GD Performance Tracker — Integration Guide

Two RAM-only instrumentation utilities for release builds: **GDPerfTracker** (FPS & memory during gameplay) and **GDStartupTime** (cold-start timing). Neither writes to disk or sends anything itself — the developer consumes a payload and logs it through the project's own analytics.

Package contents

Item	What it is
`Scripts/GDPerformance.cs`	**The ONLY class developers call** — a 4-method facade: `Configure`, `ConsumePerfPayload`, `MarkGameInteractive`, `GetStartupPayload`. Everything else in the package is internal implementation and hidden from IntelliSense.
`Prefabs/GDPerfTracker.prefab`	**Drag-and-drop setup** — ready GameObject with the recorder attached. Its Inspector holds the **initial values** for all three settings (FPS tracking off, interval 1s, startup reporting off); `Configure()` from remote config overrides them at runtime. Drop into the first scene (boot/splash); survives scene loads automatically.
`Scripts/GDPerfTracker.cs`	FPS & memory recorder implementation (Section 1). Lives on the prefab — **never called directly**.
`Scripts/GDStartupTime.cs`	Cold-start stopwatch implementation (Section 2). Needs no prefab or scene object — **never called directly**.
`Editor/PerformanceTrackerWizard.cs`	**Setup wizard** — menu `Tools → GD Performance Tracker → Performance Tracker Wizard`. Step 1 adds the prefab to a scene you pick; Step 2 shows the three integration calls with Copy buttons. Editor-only; never ships in builds.
`Documentation/GDPerformanceTracker-Guide.pdf`	This guide.

Setup flow — wizard, then three calls

Run **Tools → GD Performance Tracker → Performance Tracker Wizard**. It does the only automated step and then shows you the rest:

Step	Who	What happens
Scene setup	Wizard (Step 1)	Pick a scene (normally boot/splash) — the wizard adds the GDPerfTracker prefab to it and saves the scene. Skipped automatically if the scene already has one.
1. Configure	Developer	Once at startup, after remote config is fetched: `GDPerformance.Configure(perfOn, interval, startupOn)` — overrides the prefab's Inspector defaults. Without it, the Inspector values apply (shipped OFF).
2. Log FPS/memory	Developer	At your logging moment (level end / session end): `GDPerformance.ConsumePerfPayload()` → add adid/appToken → `LogCustomEvent("perfStats", perf)`.
3. Log startup time	Developer	`GDPerformance.MarkGameInteractive()` once, where the game becomes genuinely playable; then `GDPerformance.GetStartupPayload()` → add adid/appToken → `LogCustomEvent("loadingTime", payload)`.

The wizard's Step 2 shows the exact code for calls 1–3 with Copy buttons; full versions are in the sections below.

Section 1 — GDPerfTracker (FPS & Memory)

`GDPerformanceTracker/Scripts/GDPerfTracker.cs · MonoBehaviour (place on a bootstrap object)`

What it is — one line

A lightweight recorder that samples FPS and memory into RAM while its switch is on — **disabled by default**; the developer turns it on, and hands themselves one analytics-ready payload whenever they ask.

How to use

```
// 1. Drag the ready-made prefab into your FIRST scene (boot/splash):
//    GDPerformanceTracker/Prefabs/GDPerfTracker.prefab
//    (or run Tools → GD Performance Tracker → Performance Tracker Wizard)
//    It records NOTHING until you enable it:
GDPerformance.Configure(remoteEnabled, remoteIntervalSeconds);
// 2. Wherever YOU log your event (level end, session end — your choice):
var perf = GDPerformance.ConsumePerfPayload();    // null = nothing recorded, skip
if (perf != null) YourAnalytics.SendEvent("perfStats", perf);
```

Full example — Firebase Remote Config → tracker → Metica

```
// ---- 1) On startup, AFTER remote config has been fetched
//         (Firebase, behind MonetizationServices.Remote):
void ApplyPerfRemoteConfig()
{
    bool  perfOn    = MonetizationServices.Remote.GetRemoteValue<bool>("perf_tracking_enabled");
    float interval  = MonetizationServices.Remote.GetRemoteValue<float>("perf_sample_interval"); // 1 =
one sample/sec
    bool  startupOn = MonetizationServices.Remote.GetRemoteValue<bool>("startup_tracking_enabled");
    GDPerformance.Configure(perfOn, interval, startupOn);
    // off  ->  tracker records nothing / startup payload returns null — nothing is sent
}

// ---- 2) At YOUR logging moment (level complete, session end — your choice):
void LogPerfToMetica()
{
    Dictionary<string, object> perf = GDPerformance.ConsumePerfPayload();
    if (perf == null) return;                    // tracking off / nothing recorded — skip

    // REQUIRED on every event in our pipeline (same base fields
    // GDMeticaAnalytics.CreateBaseEvent() attaches):
    perf["adid"]     = AdjustAnalyticsNetwork.AdId;
    perf["appToken"] = AdjustAnalyticsNetwork.AppToken;

    perf["level"] = currentLevelNo;              // optional: add your own context fields

    MeticaSdk.Analytics.LogCustomEvent("perfStats", perf);
}
```

> **Required base fields:** like every event in this project, the payload must carry `adid` and `appToken` (from `AdjustAnalyticsNetwork`) — the tracker does not add them itself. **Metica note:** `LogCustomEvent` rejects Metica's core event type names (install, login, purchase, …) — the custom name `"perfStats"` is safe.

Developer API — `GDPerformance` is the only class you call

Everything else (GDPerfTracker, GDStartupTime, PerfSummary) is `internal` — hidden from IntelliSense on purpose, so the integration surface is exactly these four methods:

Method	What it does
`Configure(perfEnabled, interval, startupEnabled)`	Both kill switches + sample rate, typically from remote config. `perfEnabled = false` → FPS/memory tracker stops sampling and clears data. `startupEnabled = false` → `GetStartupPayload()`

	returns null (no loadingTime event). Interval = seconds per FPS sample (clamped 1–30, default 1). Everything ships **OFF** — until this runs, nothing records or logs.
`ConsumePerfPayload()`	Analytics-ready dictionary of everything recorded since the last consume, then resets the window (recording continues). Returns **null** when tracking is off or nothing was recorded — skip logging on null.
`MarkGameInteractive()`	Call ONCE where this game becomes genuinely playable. Idempotent — extra calls are safe no-ops. Unity-control time is captured automatically; nothing to call for it.
`GetStartupPayload()`	Cold-start dictionary for the `loadingTime` event: `unityControlMs`, `interactiveMs` (-1 = not measured). Returns **null** when startup tracking is disabled — skip logging on null.

> **Both features have kill switches, both OFF by default — configurable in two layers. Layer 1, Inspector:** the prefab exposes initial values for all three settings (FPS tracking on/off, sample interval, startup reporting on/off) — what ships in the build. **Layer 2, remote config:** `Configure(perfEnabled, interval, startupEnabled)` overrides the Inspector values at runtime when it runs. Startup milestones are still captured in RAM while disabled — two timestamp reads, effectively free — so enabling later (when remote config arrives) still has the data; while disabled, `GetStartupPayload()` returns null so nothing gets logged.

Payload fields — FPS

FPS is sampled in buckets (default 1 per second). All values are whole numbers.

Field	Meaning & how to use it
`fps_avg`	Average FPS over the window. Headline number; it hides sudden slowdowns — never judge by this alone.
`fps_min`	Single worst sample bucket. Shows the deepest dip that occurred.
`fps_p01`	**1% low — measures the player's worst moments.** The FPS the game runs at during its roughest seconds. The most important field: a game with avg 40 but p01 of 8 feels broken to the player.
`fps_p05`	5% low. Shows whether slowdowns happen often: if p05 is far below p50, drops are frequent, not one-off.
`fps_p25`	Lower-quartile FPS. The "bad quarter" of the session.
`fps_p50`	Median — the typical experience. More honest than avg (not skewed by extremes).
`fps_p75`	Upper quartile. The "good three-quarters" boundary.
`fps_p95`	Near-best FPS. Compare with the cap: p95 at the cap = device has headroom.
`fps_p99`	Best sustained FPS reached. p99 far under the cap = device can never reach target.
`worst_ms`	Slowest single frame in milliseconds. One 400ms frame = a short visible freeze even if the FPS numbers look fine. Watch this to catch one-moment freezes (saving, object spawning, effect loading).

> **Reading FPS against the cap:** always compare with `tier_fps`. On a 30-capped device, p50 = 29 is perfect; on a 60-capped device, p50 = 29 is a problem. Healthy profile: p50 near the cap, p01 not far below p50.

Payload fields — Memory (all in MB)

Field	Meaning & how to use it
`ram_total`	The device's physical RAM. Denominator for everything below — 900 MB peak is fine on 8 GB, dangerous on 3 GB.
`mem_alloc_peak / _avg`	**Allocated** — memory Unity is actively using (textures, meshes, audio, managed heap). Rises = assets/leaks growing.
`mem_resv_peak / _avg`	**Reserved** — memory Unity has claimed from the OS (allocated + internal pools). Always ≥ allocated.

`mem_sys_peak / _avg`	**System** — whole process footprint as the OS sees it (Unity + plugins + ad SDKs). Closest to what Android's low-memory killer judges. Rule of thumb: peak above ~25–30% of `ram_total` risks background kills.

Payload fields — Context

Field	Meaning
`seconds`	How long this window recorded. Short windows (<30s) have noisy percentiles — weigh accordingly.
`tier_fps`	The FPS cap active during recording (`Application.targetFrameRate`). Required to interpret every FPS field fairly.

Performance cost of the tracker itself

Per frame: a handful of float operations (~nanoseconds). Memory counters are values Unity already maintains — reading them is free. Buffers cap at ~30 KB of RAM. The only measurable work is one sort of ≤4,096 floats at `ConsumePayload()` — under a millisecond, once per consume. Safe to ship enabled for all players; the remote kill switch can stop it fleet-wide without an update.

Section 2 — GDStartupTime (Cold-Start Timing)

`GDPerformanceTracker/Scripts/GDStartupTime.cs · static class (no component, no setup)`

What it is — one line

A cold-start stopwatch that measures from **actual OS process creation** (not Unity engine init — so it includes APK/dex load and native splash time) to two milestones: Unity taking control (captured automatically) and the game becoming playable (one developer call).

How to use

```
// 1. Script is in the project — Unity-control time is captured automatically,
//    nothing to call for that milestone. No prefab or scene object needed.
// 2. Fire once, the moment THIS game is genuinely playable
//    (menu ready / first input enabled / first level loaded — per-game decision):
GDPerformance.MarkGameInteractive();
// 3. When building the event to log:
Dictionary<string, object> payload = GDPerformance.GetStartupPayload();
// payload = { "unityControlMs": 842f, "interactiveMs": 2310f }
```

How the kill switch works despite remote config arriving late: cold start happens BEFORE Firebase can answer "is tracking allowed?" — so waiting for the flag would mean there is nothing left to measure. Instead, milestones are always captured into RAM (two timestamp reads, zero cost, nothing leaves the device), and the kill switch gates the **output**: while disabled, `GetStartupPayload()` returns null so no event is logged. Measure always, report only when allowed.

Sending it to Metica

```
Dictionary<string, object> payload = GDPerformance.GetStartupPayload();

// Required base fields — Metica needs both to correlate cold-start
// performance with attribution/campaign data:
payload["adjustAdid"]     = adjustAdid;
payload["adjustAppToken"] = adjustAppToken;

MeticaSdk.Analytics.LogCustomEvent("loadingTime", payload);
```

Null warning: `adid` can legitimately still be null this early in a cold start. Run the payload through the existing null-stripping `DictionaryExtensions` helper before it reaches Metica — the SDK is known to silently drop the *entire event* on iOS if any property value is null.

Payload fields

Field	Type	Meaning
`unityControlMs`	float	Process start → Unity's managed code takes over (ms). Captured automatically.
`interactiveMs`	float	Process start → game genuinely playable (ms). Captured by the developer's `CaptureInteractiveTime()` call.

-1 means "not measured": a field comes back as `-1` (NotCaptured) when its capture never ran — deliberately, so a missing capture can't masquerade as a great 0ms time. If you see -1 in dashboards, check the integration at that call site.

Notes & caveats

- **Idempotent:** only the first call per session is recorded — an accidental second `CaptureInteractiveTime()` (e.g. re-firing on a tutorial-skip path) is a safe no-op.
- **Android API 24+:** uses `Process.getStartUptimeMillis()` + `SystemClock.uptimeMillis()` — same clock base, both exclude deep sleep, so the delta is clean awake-time.
- **Editor / pre-API-24 fallback:** falls back to `Time.realtimeSinceStartup`, which measures from engine init, not process creation — those readings are **not comparable** to real Android numbers; flag them if they land in the same dashboards.
- **Android only for now** — no iOS branch yet. Add one before relying on this for any iOS title in the portfolio.